Input-Output Organization

Dr. V S M Srinivasavarma

Outline

- Peripheral Devices
- Input-output Interface
- Priority Interrupt
- Direct Memory Access
- Serial Communication

Peripheral Devices – Input Devices

- Keyboard
- Optical Input Device
  - Card reader
  - Paper tape reader
  - Bar code reader
  - Digitizer
  - Optical Mark reader
- Magnetic Input Device
  - Magnetic Stripe Reader
- Screen Input Device
  - Touch Screen
  - Light Pen
  - Mouse
- Analog Input Device

Peripheral Devices – Output Devices

- Card Puncher
  - Paper Tape Puncher
- CRT
- Printer
  - Impact, Ink Jet, Laser, Dot Matrix
- Plotter
- Analog
- Voice - Speakers

Input – Output Interfaces

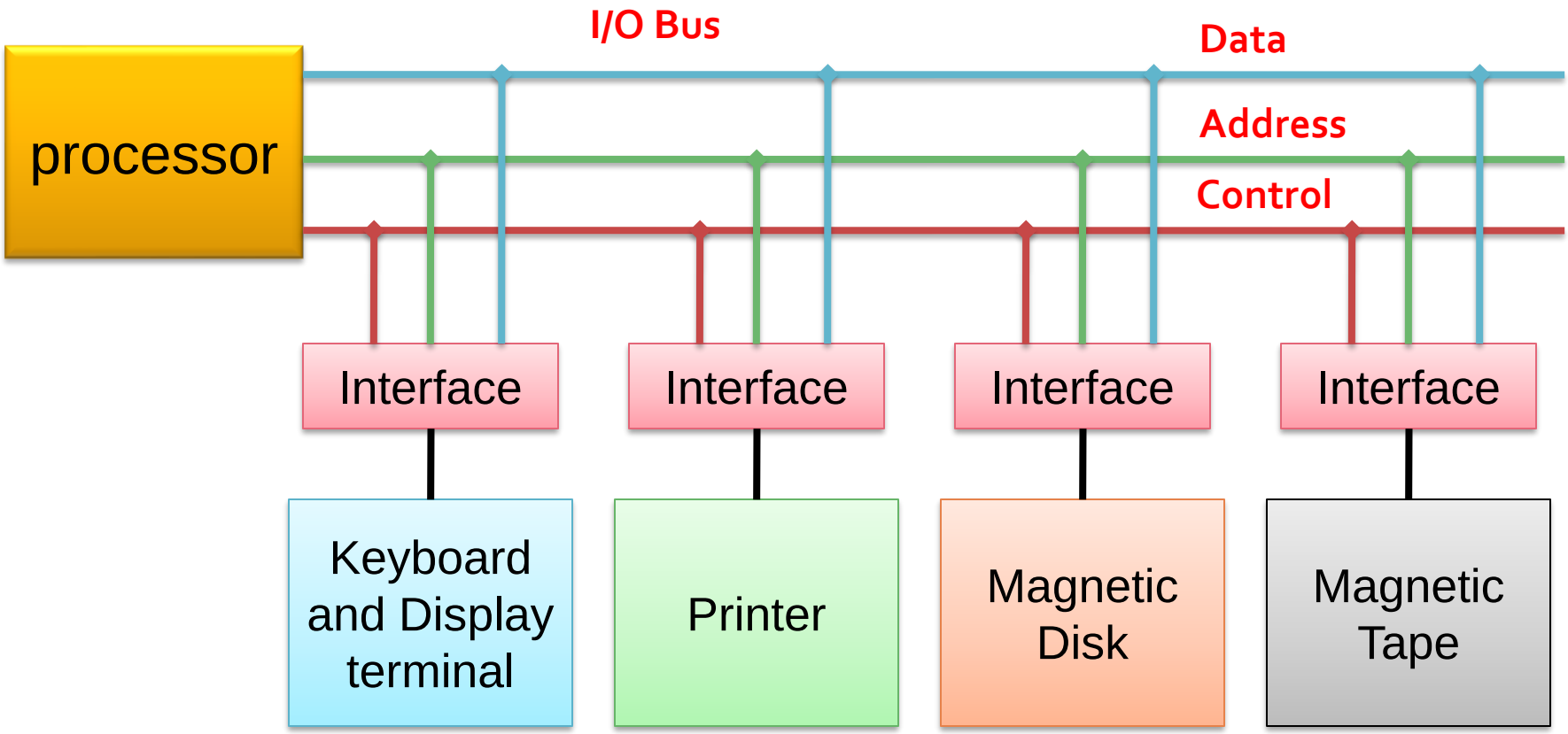
- Provides a method for transferring information between internal storage (Such as Memory and CPU registers) and external I/O devices
- Resolves the differences between the computer and peripheral devices
  - Peripherals – Electromechanical Devices
  - CPU or Memory – Electronic devices
- Data Transfer Rate
  - Peripherals – usually slower
  - CPU or Memory – usually faster than peripherals
  - Some kinds of synchronization mechanism may be needed

Input – Output Interfaces

- Unit of Information
  - Peripherals – Byte
  - CPU or Memory - Word
- Operating Modes
  - Peripherals – Autonomous, Synchronous
  - CPU or Memory - Synchronous

I/O Bus

- Communication with in a computer via a single shared bus

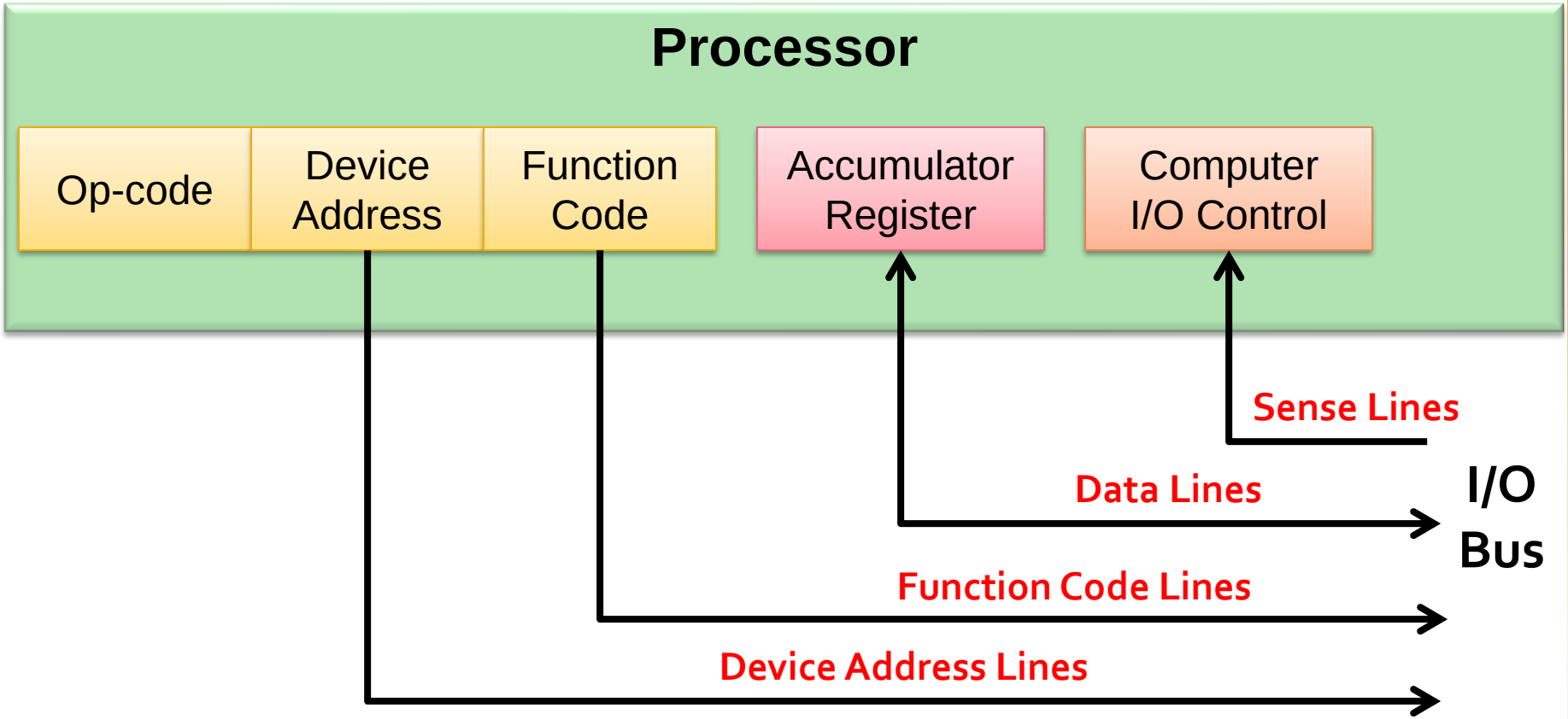


Interface Module

- Each peripheral has an interface module associated with it
- Interface
  - Decodes the device address (device code)
  - Decodes the commands (operation)
  - Provides signals for the peripheral controller
  - Synchronizes the data flow and supervises the transfer rate between peripheral and CPU or Memory
- Typical I/O Instruction

Op-code	Device Address	Function Code
		Command

Connection of I/O Bus to CPU



Types of Commands

- Control Command
  - Activates the peripheral and inform it what to do
- Status Command
  - Test various status conditions in the interface or the peripherals
- Data-output Command
  - Output
- Data-input Command
  - Input

I/O Bus and Memory Bus: Functions

- Memory Bus
  - Is for information transfer between CPU and the main memory
- I/O Bus
  - Is for information transfer between CPU and I/O devices through their I/O interface
- Physical Organizations
  - Many computers uses a common single bus system for both memory and I/O interface units
  - 2 – Way bus:
    - Use one common bus but separate control lines for each function
    - Use one common bus with common control lines for both functions
  - Some computer systems use two separate buses, one to communicate with memory and the other with I/O Interface

I/O Bus

- Communication between CPU and all interface units is via a common I/O bus
- An interface connected to a peripheral device may have a number of data registers, a control register, and a status register
- A command is passed to the peripheral by sending to the appropriate interface register
- Function code and sense lines are not needed
  - Transfer of data, control and status information is always via the common I/O Bus

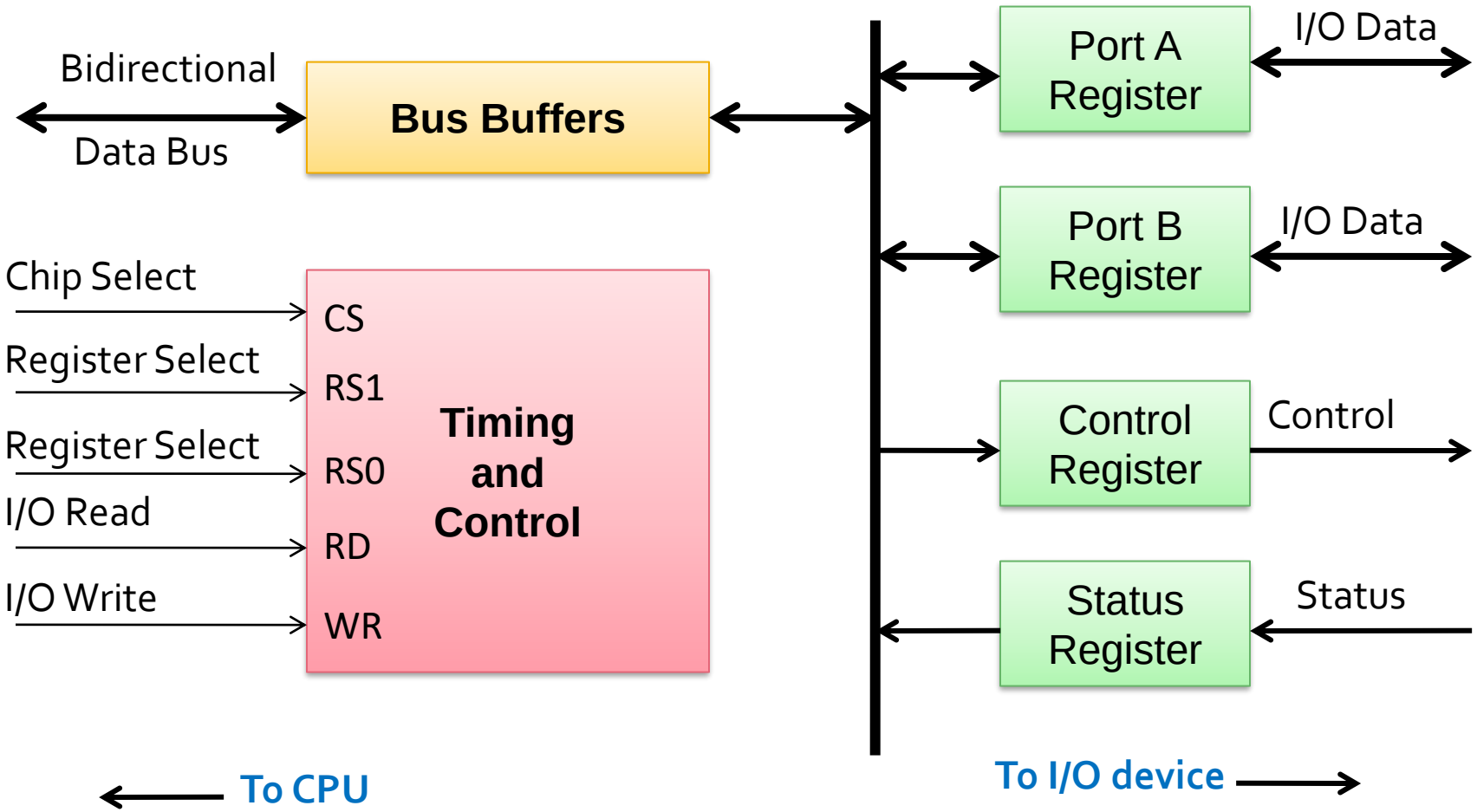
Isolated I/O (I/O Mapped I/O)

- Separate I/O read/write control lines in addition to memory read/write control lines
- Separate (isolated) memory and I/O address spaces
- Distinct input and output instructions
  - IN and OUT Instructions

Memory Mapped I/O

- A single set of read/write control lines
  - No distinction between memory and I/O transfer
- Memory and I/O addresses share the common address space
  - Reduces memory address range available
- No specific input or output instruction
  - The same memory reference instructions can be used for I/O transfers
- Considerable flexibility in handling I/O operations
  - All instruction directly access memory location and can manipulate

Programmed I/O Interface



Programmed I/O Interface

CS	RS1	RS0	Register Selected
0	X	X	None – data bus in High impedence
1	0	0	Port A Register
1	0	1	Port B Register
1	1	0	Control Register
1	1	1	Status Register

Programmable Interface

- Information in each port can be assigned a meaning depending on the mode of operation of the I/O device
  - Port A = Data; Port B = Command; Port C = Status
- CPU initializes (loads) each port by transferring a byte to the Control Register
  - Allows CPU can define the mode of operation of each port
  - Programmable Port
    - By changing the bits in the control register, it is possible to change the interface characteristics

Data Transfer

- The data transfer between the CPU and the peripherals can be two ways
- Synchronous
  - All devices derive the timing information from common clock line
- Asynchronous
  - No common clock
- Asynchronous Data Transfer
  - Asynchronous data transfer between two independent units requires that control signals to be transmitted between the communicating units
    - To indicate the time at which data is being transmitted

Two Asynchronous Data Transfer Methods

- Strobe Pulse:
  - A strobe pulse is supplied by one unit to indicate the other unit when the transfer has to occur
- Handshaking:
  - A control signal is accompanied with each data being transmitted to indicate the presence of data
  - The receiving unit responds with another control signal to acknowledge receipt of the data

DMA (Direct Memory Access)

- Large blocks of data transferred at a high speed to or from high speed devices
  - Magnetic drums, disks, tapes, etc.,
- DMA Controller
  - Interface that provides I/O transfer of data directly to and from the memory and the I/O device
- CPU initializes the DMA controller by sending a memory address and the number of words to be transferred
- Actual transfer of data is done directly between the device and memory through DMA controller
  - Freeing CPU for other tasks

Interrupts

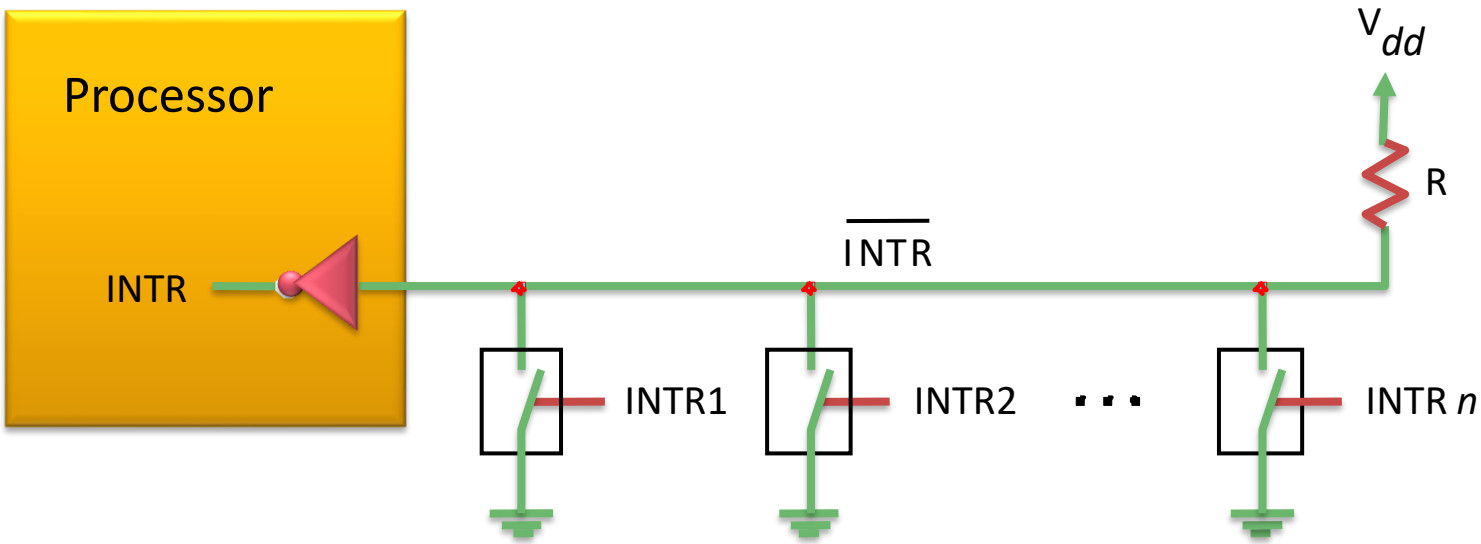
- Interrupt Request Line
  - Interrupt-request signal
  - Interrupt-acknowledge signal
- Interrupt Service Routine
  - Similar to subroutine
  - May have no relationship to program being executed at time of interrupt
    - Program info must be saved
    - Interrupt latency

Handling Interrupts

- Many situations where the processor should ignore interrupt requests
  - Interrupt-disable
  - Interrupt-enable
- Typical scenario
  - Device raises interrupt request
  - Processor interrupts program being executed
  - Processor disables interrupts and acknowledges interrupt
  - Interrupt-service routine executed
  - Interrupts enabled and program execution resumed

Interrupt from N Devices

- An equivalent circuit for an open-drain bus used to implement a common interrupt-request line.



Handling Multiple Devices

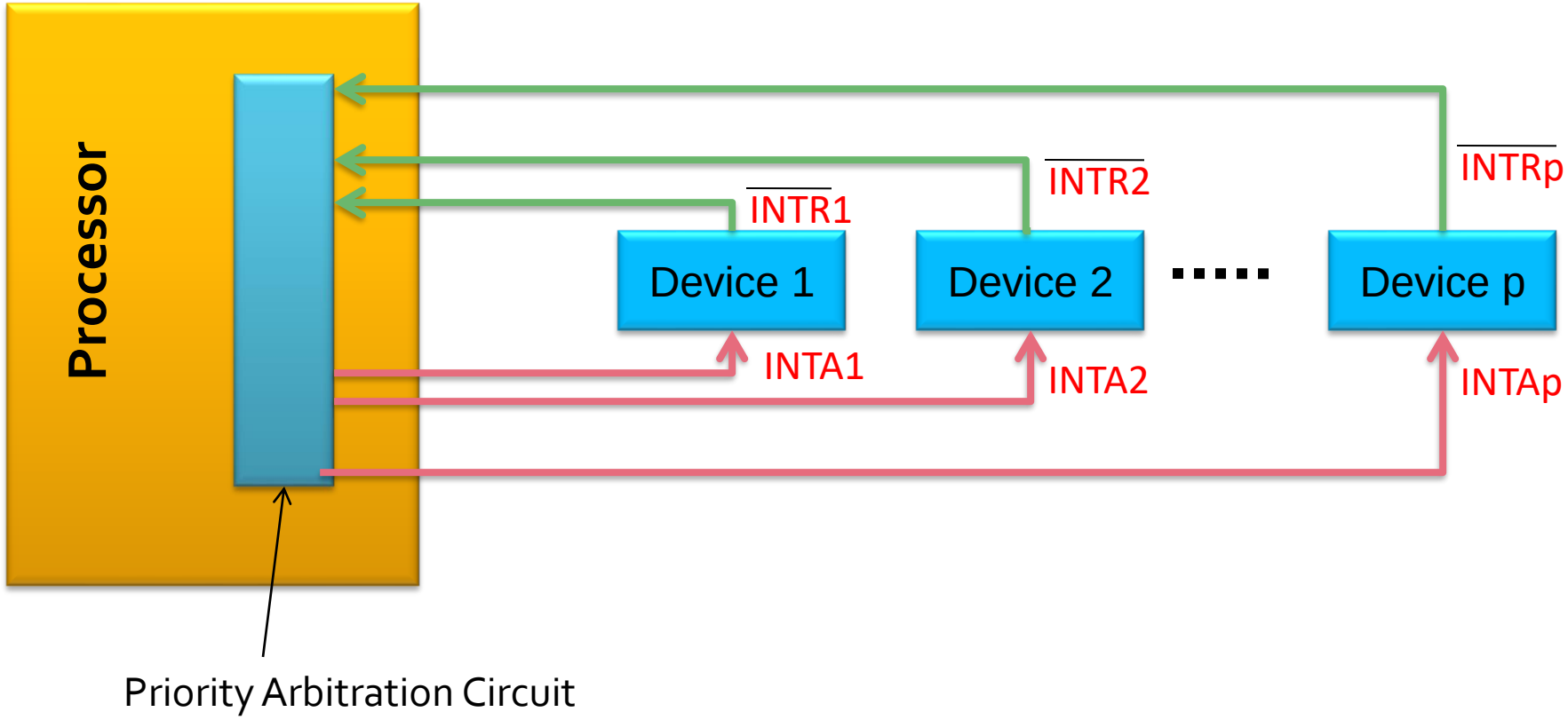
- How can the processor recognize the device requesting an interrupt?
- How can the processor obtain the starting address of the appropriate interrupt-service routine?
- Should a device be allowed to interrupt the processor while another interrupt is being serviced?
- How should two or more simultaneous interrupt requests be handled?

Interrupt Priority

- Multi-level priority organization
- During execution of interrupt-service routine
  - Disable interrupts from devices at the same level priority or lower
  - Continue to accept interrupt requests from higher priority devices
  - Privileged instructions executed in supervisor mode
- Controlling device requests
  - Interrupt-enable
    - EN, DEN

Interrupt Priority

- Implementation of interrupt priority using individual interrupt-request and acknowledge lines



Interrupt Priority

- Polled interrupts
  - Priority determined by the order in which processor polls the devices (polls their status registers)
- Vectored interrupts
  - Priority determined by the order in which processor tells device to put its code on the address lines (order of connection in the chain)

Priority Interrupt

- Determines which interrupt is to be served first when two or more requests are made simultaneously
- Also determines which interrupts are permitted to interrupt the computer while another is being serviced
- Higher priority interrupts can make requests while servicing a lower priority interrupt

Priority Interrupt by Software (Polling)

- Priority is established by the order of polling the devices (interrupt sources)
- Flexible since it is established by software
- Low cost since it needs a very little hardware
- Very slow

Priority Interrupt by Hardware

- Require a priority interrupt manager which accepts all the interrupt requests to determine the highest priority request
- Fast since identification of the highest priority interrupt request is identified by the hardware
- Fast since each interrupt source has its own interrupt vector to access directly to its own service routine

Direct Memory Access (DMA)

- Polling or interrupt driven I/O incurs considerable overhead
  - Multiple program instructions
  - Saving program state
  - Incrementing memory addresses
  - Keeping track of word count
- Transfer large amounts of data at high speed without continuous intervention by the processor
- Special control circuit required in the I/O device interface, called a DMA controller
- DMA controller keeps track of memory locations, transfers directly to memory (via the bus) independent of the processor

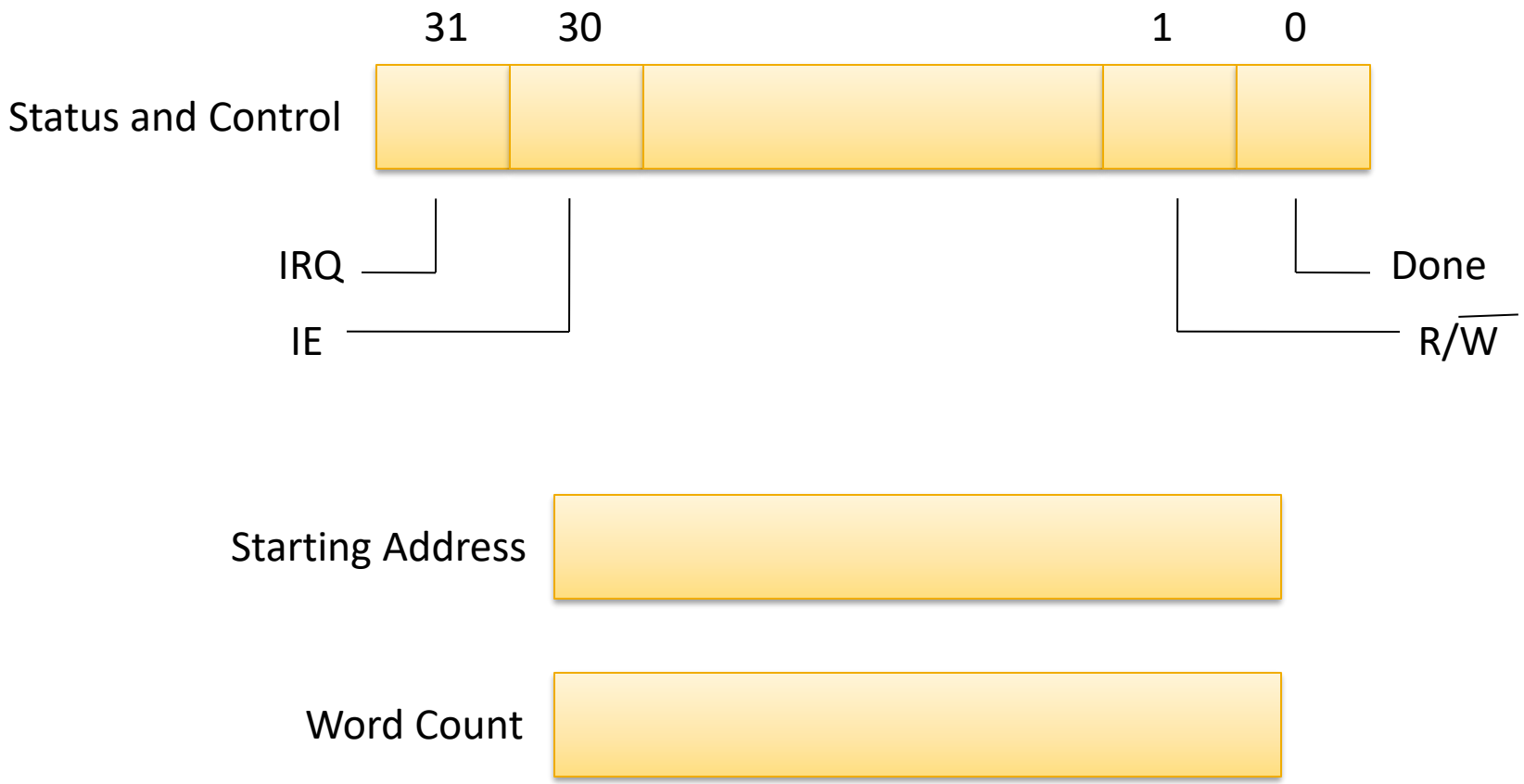
DMA Controller

- Part of the I/O device interface
  - DMA Channels
- Performs functions that would normally be carried out by the processor
  - Provides memory address
  - Bus signals that control transfer
  - Keeps track of number of transfers
- Under control of the processor

Direct Memory Access (DMA)

- OS responds to a program's system call for Disk Read, for example
- OS puts the program in the "blocked" or "waiting" or "asleep" state
- Initiates the Disk Read
- Starts execution of another program
- When Read is completed, DMA controller sends an interrupt

Registers in a DMA interface

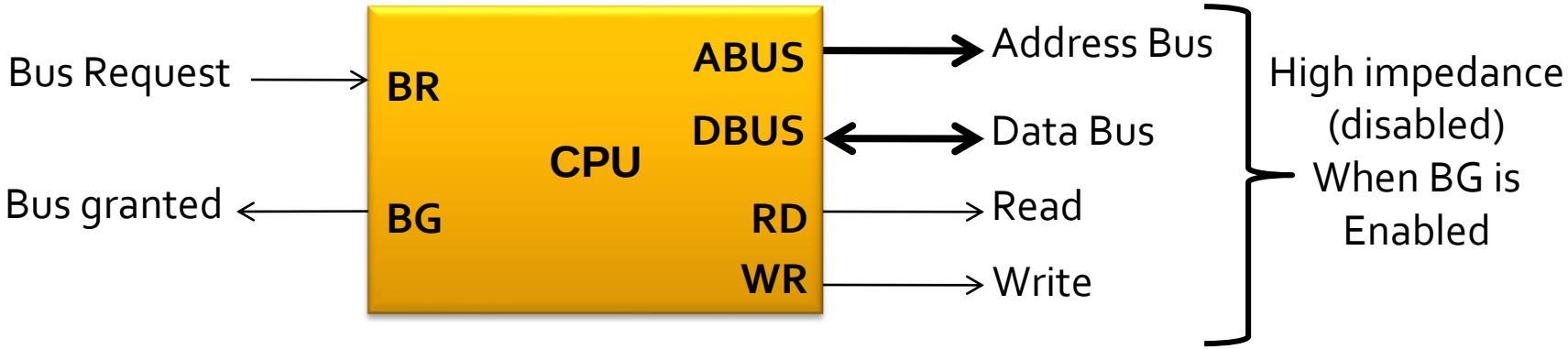


Sharing the Bus

- Bus master: the device currently allowed to transfer data on the bus (CPU or DMA)
- Bus arbitration: choosing the next bus master
- Bus busy: BBSY set by current bus master to show that the line is in use
- Bus request: BR set when device wants to become bus master
  - When it gets BG (bus granted), waits for BBSY to become inactive, then assumes mastership of the bus and sets BBSY

Direct Memory Access

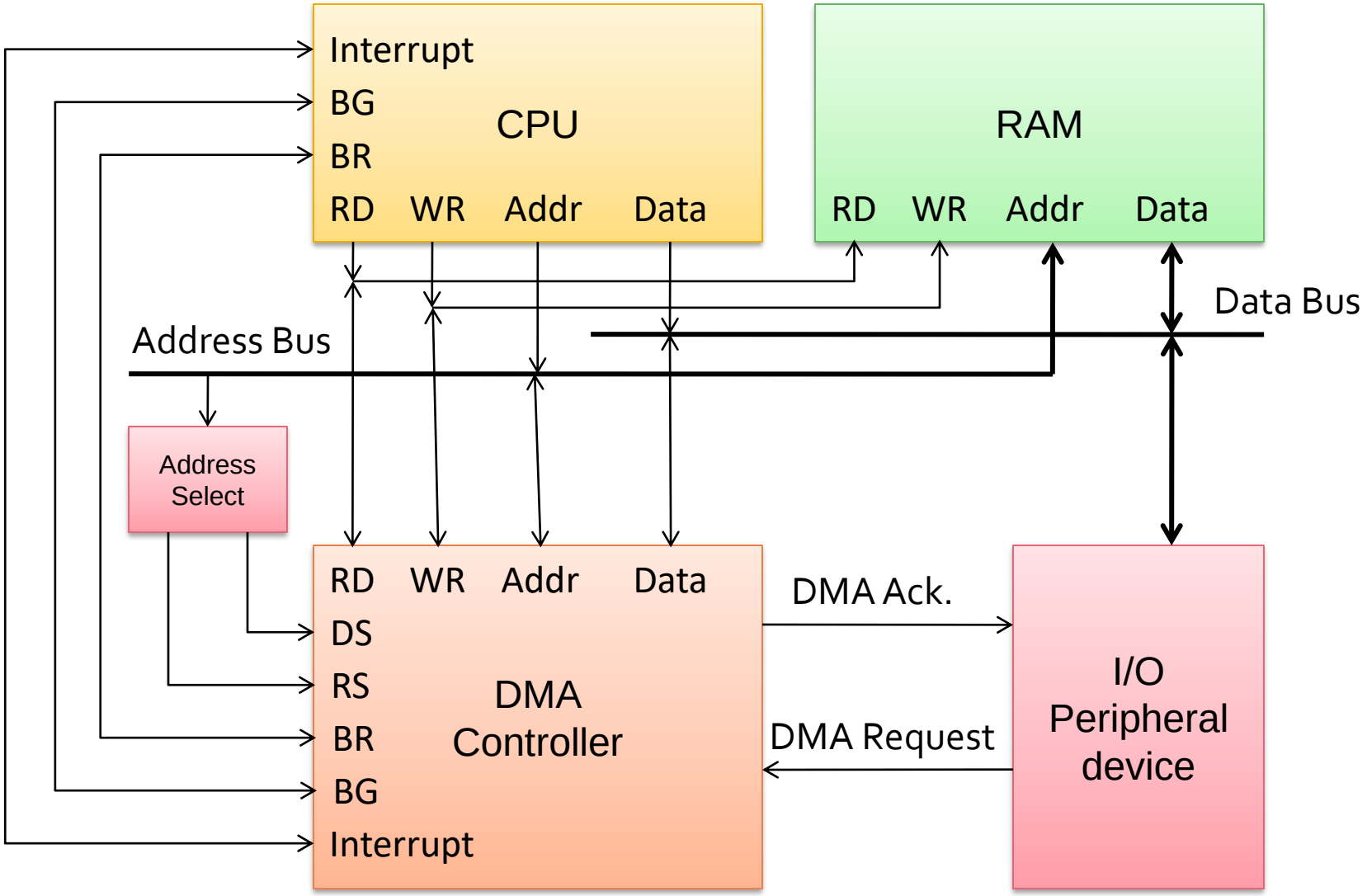
- Block of data transfer from high speed device
  - Drum, Disk, Tape
- DMA Controller
  - Interface which allows I/O transfer directly between Memory and Device
  - Freeing CPU for other tasks
- CPU initializes DMA controller by sending memory address and the block size (number of words)



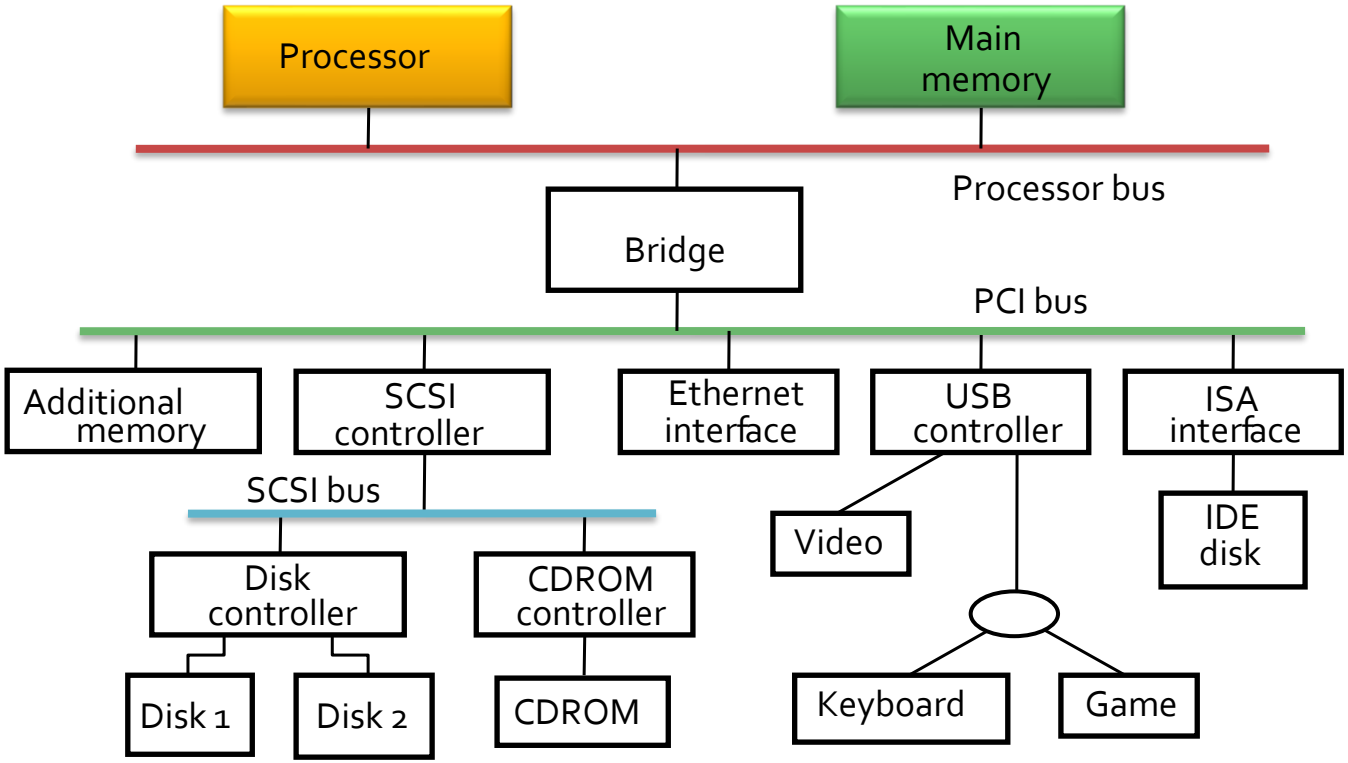
DMA Vs CPU Memory Access

- While DMA I/O takes place, CPU is also executing instructions
  - DMA controller and CPU both access Memory
    - Memory Access Conflict
- Memory Bus Controller
  - Coordinating the activities of all devices requesting memory access
  - Priority System
  - Memory access by CPU and DMA controller are interwoven, with the top priority given to DMA controller
    - Cycle Stealing

DMA Transfer



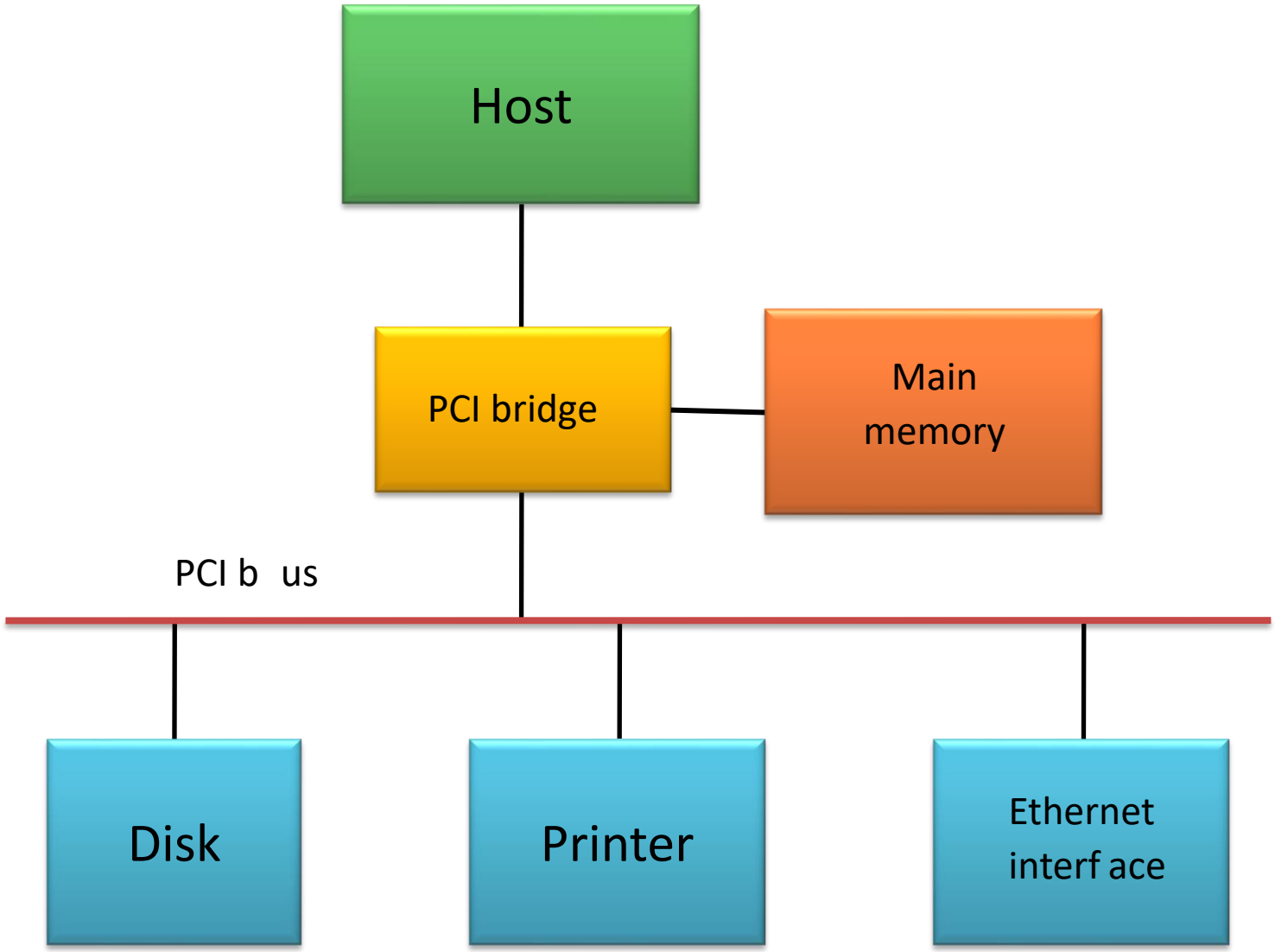
Computer System With Different Interface Standards



System Buses

- Bridge: Connects buses with different characteristic
  - items on PCI bus appear as though on the system bus
- PCI: Peripheral Component Interconnect
  - Processor independent
- SCSI: Small Computer System Interface
  - SCSI controller uses DMA approach
- USB: Universal Serial Bus
- ISA: Industry Standard Architecture
- EISA: 32 bit version of ISA
- IDE: Integrated Device Electronics

PCI Bus in a Computer



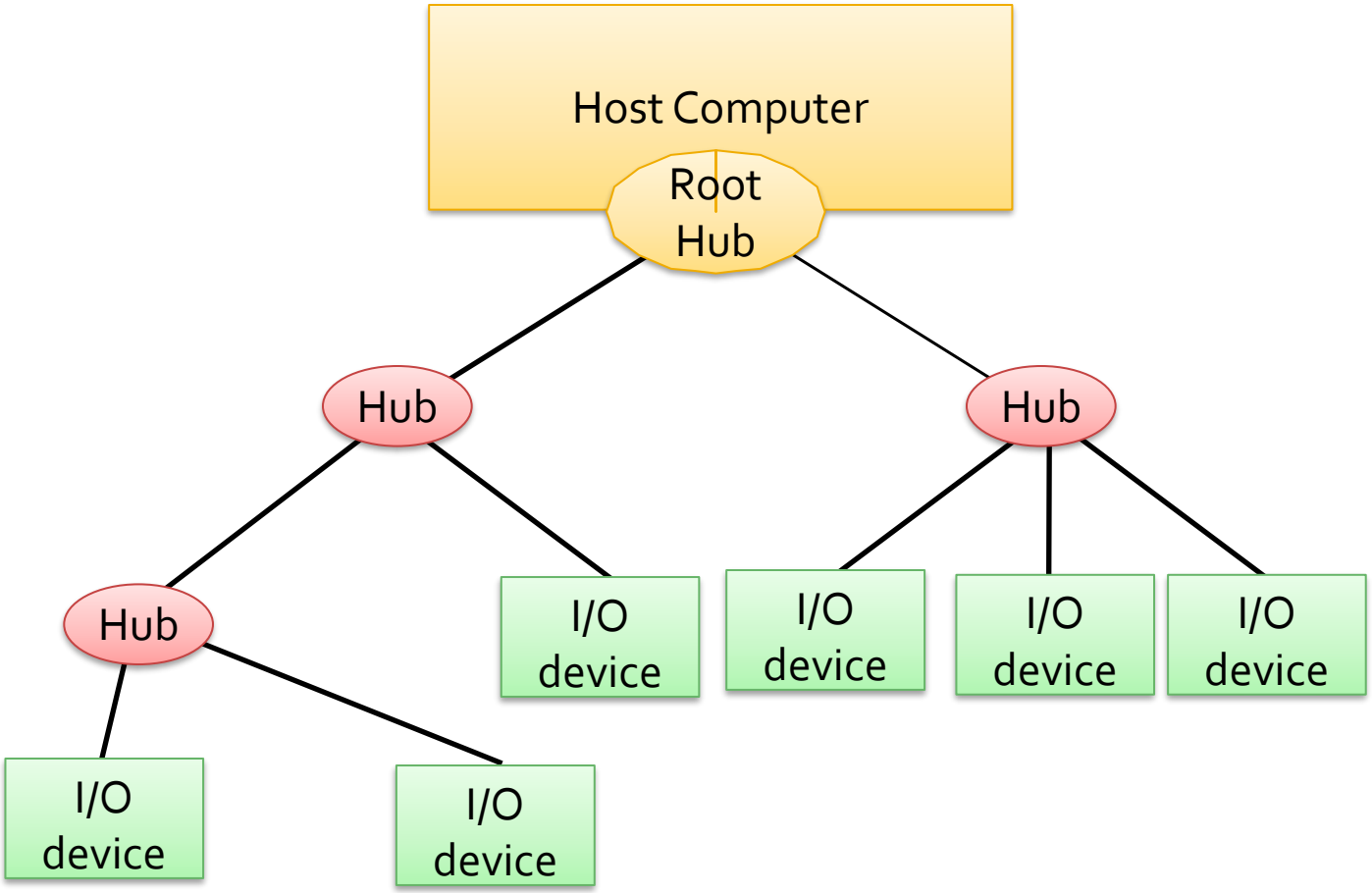
USB Interface

- Need for high transfer rates
- Need devices at some distance
- High rate ➔ wide bus?
  - Problem over distance because of skew
- Serial transmission
  - Low cost and flexible
  - No skew, so high rates OK, distance OK

USB Interface

- Hubs have multiple ports
- Devices connect to a port
- Hubs connect to a port
- “Tree structure”

Universal Serial Bus tree structure



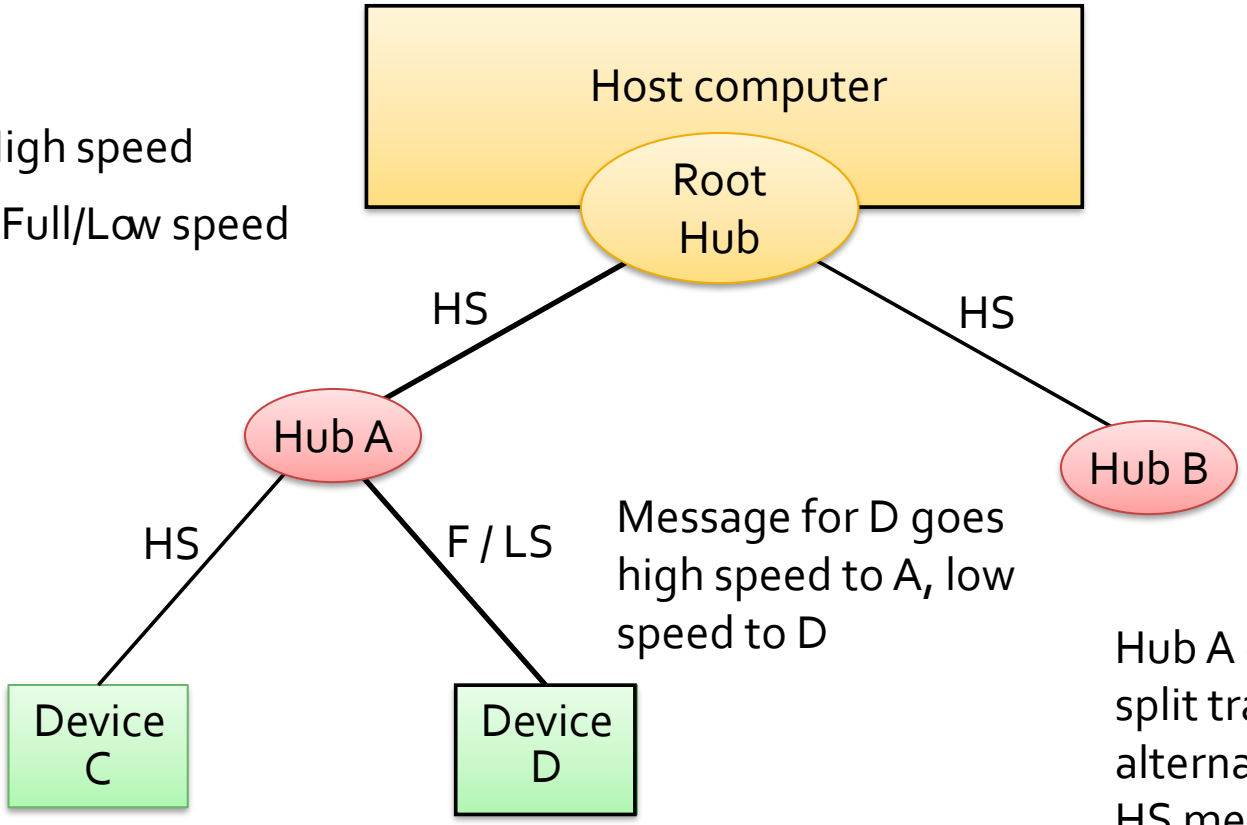
USB Interface

- Message from host copied to all hubs, to all ports, to all devices
- Only the addressed device responds
- Message from device only goes “upstream”--up the tree
- Makes possible a large number of devices through a few ports at the root hub.
- High speed, long distances possible

Split bus operation

HS - High speed

F / LS - Full/Low speed



Message for D goes high speed to A, low speed to D

Hub A can operate in split traffic mode, alternately sending HS messages to C while sending parts of LS message to D

USB

- Data is transmitted in packets of one or more bytes
- First field is always a PID (packet id)
  - A 4 bit ID plus the complement for verification
  - Therefore, 16 different packet types
- Control packets such as ACK are PID only
- Token packets used for control
- Data packets

USB Packet Format

PID_0	PID_1	PID_2	PID_3	$\overline{PID_0}$	$\overline{PID_1}$	$\overline{PID_2}$	$\overline{PID_3}$

(a) Packet identifier field

8 bits	7 bits	4 bits	5 bits
PID	ADDR	ENDP	CRC16

(b) Token packet, IN or OUT

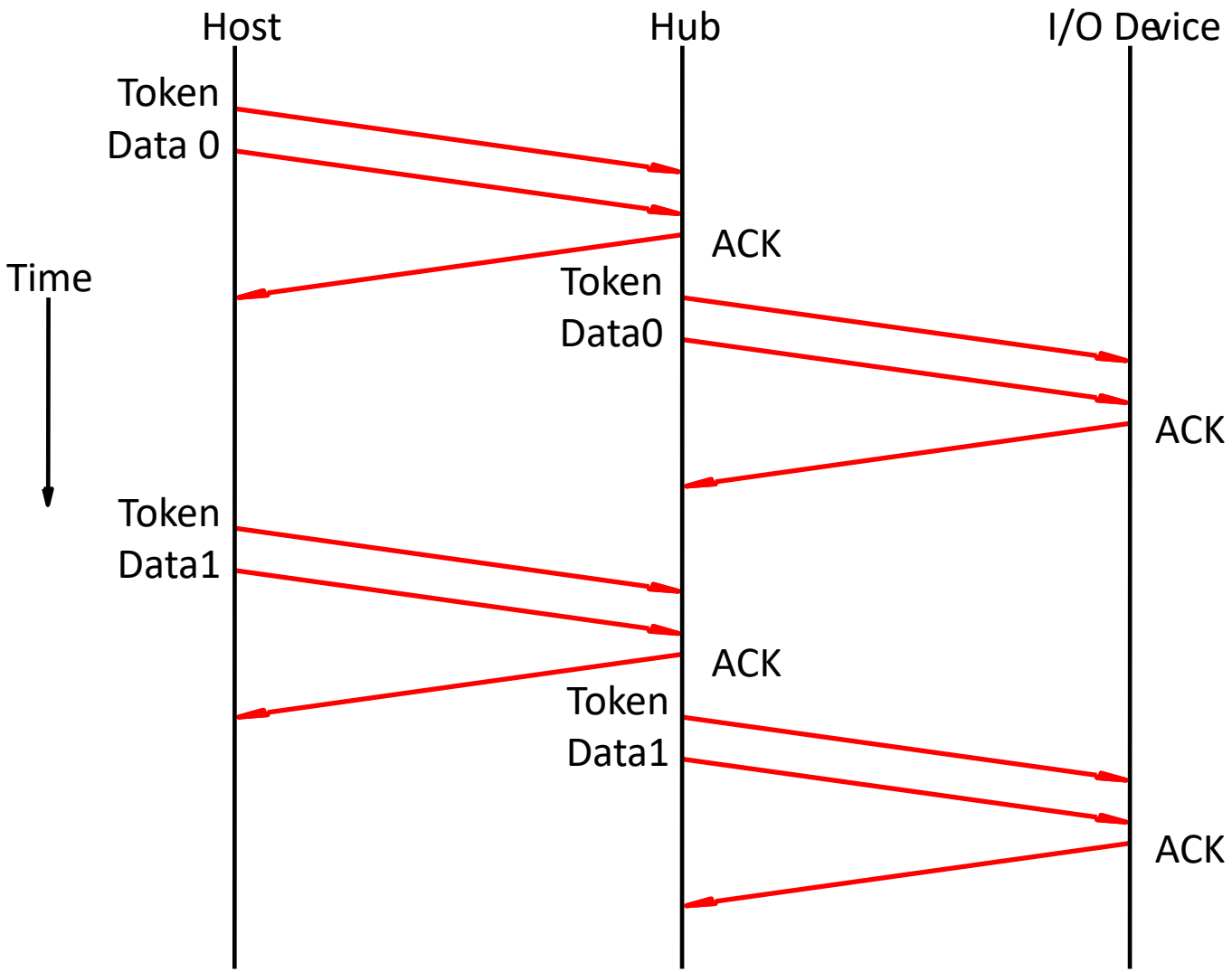
8 bits	0 to 8192 bits	16 bits
PID	DATA	CRC16

(c) Data packet

USB Packet Format

- ADDR: The address, a 7 bit device address (address is local to the USB tree)
- ENDP: The endpoint, the location within the device (status register, data register, etc.)
- Data packet follows a token packet containing the address and endpoint of the device.
- Packet IDs contain bit that alternates between 0 and 1 to guard against lost packets

An output transfer



USB

- Software (in host computer) sends packets to devices (hubs just forward them)
- Host software periodically polls each hub
  - New device when connected has address 0
  - If a new device has been added, host collects info about the device from the memory of the device and *assigns* it a 7 bit address
- Locations within the device (registers) are identified by a 4 bit endpoint address

Reference

- **Chapter 4: Computer Organization** by Carl Hamacher, Zvonko Vranesic and Safwat Zaky, 5 edition, McGraw Hill Publications.
- **Chapter 11: Computer Systems Architecture** by M Morris Mano, 3 edition, Pearson Publications.

Thank You!!